Internet of Things (IoT) with Applications

Undergraduate Engineering Course

Module 3

IoT Platforms, Edge and Cloud

Hardware Platforms, Cloud Architecture, Edge Computing, and IoT + AI

Unit III: IoT Platforms, Edge and Cloud

Concepts

Topics Covered:

Hardware Platforms: Arduino, ESP32, Raspberry Pi, NodeMCU
Cloud Computing for IoT: Architecture, Device Management, Storage, Analytics, Dashboards
Edge Computing: Edge vs Cloud, Real-Time Processing, Edge AI, Fog Computing
IoT + AI: Anomaly Detection, Predictive Maintenance, Computer Vision

1 Introduction

Modules 1 and 2 established what an IoT system is built from and how its devices talk to one another. This module addresses a different question: once data has been sensed and transmitted, *where* is it actually computed on, and *what* runs that computation? The answer spans a spectrum – from a few kilobytes of program memory on a microcontroller sitting directly next to a sensor, to elastic cloud data centres capable of training machine-learning models over years of accumulated history.

This module works through that spectrum in order. Section 2 covers the physical **hardware platforms** on which IoT logic actually runs – Arduino, ESP32, Raspberry Pi, and NodeMCU. Section 3 covers **cloud computing for IoT**: the architecture, device management, storage, and analytics services that a deployment relies on once data leaves the local network. Section 4 examines **edge computing** and the question of *where* to place processing along this spectrum, including the intermediate **fog computing** tier. Section 5 closes with **IoT + AI**: how machine learning is applied to sensor data for anomaly detection, predictive maintenance, and computer vision, and how these techniques are distributed across the edge-to-cloud spectrum developed in the earlier sections.

2 IoT Hardware Platforms

2.1 Arduino

Arduino is an open-source microcontroller platform built around a simple, beginner-friendly programming environment. It executes a single fixed program in a loop (no operating system), which keeps it lightweight, predictable, and low-power, but limits it to relatively simple control logic and modest on-board connectivity – most base Arduino boards have no built-in Wi-Fi or Bluetooth and require an additional shield or module to connect to a network.

2.2 ESP32

The **ESP32** is a low-cost microcontroller with *built-in* Wi-Fi and Bluetooth, making it one of the most widely used platforms for network-connected IoT nodes. It offers substantially more processing power and memory than a basic Arduino board while remaining inexpensive and power-efficient enough for battery-powered deployment, and is the platform used in most of this course's networked laboratory exercises (Section 6).

2.3 NodeMCU

NodeMCU refers both to an open-source firmware and to development boards built around the ESP8266 (and, in some variants, ESP32) system-on-chip, pre-packaged with built-in Wi-Fi and a Lua- or Arduino-IDE-compatible programming environment. NodeMCU boards occupy a similar niche to the ESP32 – inexpensive, Wi-Fi-enabled microcontroller development – and are popular in hobbyist and educational IoT projects for their low cost and simple USB-based programming workflow.

2.4 Raspberry Pi

The **Raspberry Pi** is a full single-board *computer* rather than a microcontroller: it runs a complete Linux operating system, supports multitasking, and can run substantially heavier software – including, notably, local machine-learning inference – that a microcontroller cannot. This added

capability comes at the cost of higher power consumption and a more complex boot/software stack than Arduino, ESP32, or NodeMCU, making the Raspberry Pi better suited to gateway or edge-processing roles (Section 4) than to simple, battery-powered sensing nodes.

Platform	**Type**	**Built-in Connectivity**	**Power Use**	**Typical Role**
Arduino	Microcontroller	None (needs shield)	Very low	Simple sensor/actuator control
ESP32	Microcontroller	Wi-Fi, Bluetooth	Low	Networked sensor nodes
NodeMCU	Microcontroller	Wi-Fi	Low	Hobbyist/educational Wi-Fi nodes
Raspberry Pi	Single-board computer	Wi-Fi, Bluetooth, Ethernet	Moderate–high	Gateway, edge processing, local AI

Table 1: Comparison of common IoT hardware platforms.

2.5 Sensors and Modules

Each of these platforms is extended through **modules** – small breakout boards that package a sensor or actuator with the supporting circuitry needed to interface it directly with a microcontroller's pins, such as the DHT11/DHT22 temperature-and-humidity module, the HC-SR04 ultrasonic distance module, or a relay module for switching mains-powered actuators. Choosing hardware for a deployment is therefore really two decisions: which processing platform (Table 1) matches the application's computation and connectivity needs, and which sensor/actuator modules match its physical measurement and control needs.

Worked Example

Selecting a hardware platform. A hobbyist project needs to run a small on-device image-classification model locally (without a cloud connection) to detect whether a parking spot is occupied, using a camera module. Which of the four platforms in Table 1 is appropriate, and why would an ESP32 alone likely be insufficient?

Solution. The Raspberry Pi is the appropriate choice: running a local image- classification model requires the general-purpose computing power and full operating system the Raspberry Pi provides. A base ESP32, while capable of very lightweight inference, has far less memory and processing power than a Raspberry Pi and would likely struggle to run a full image-classification model at usable speed and accuracy.

3 Cloud Computing for IoT

Once data reaches a gateway, most deployments forward it to a **cloud platform** for storage, longer-term analytics, and remote access – capabilities that would be impractical to replicate on-premises at the same scale and reliability.

3.1 IoT Cloud Architecture

A typical IoT cloud platform provides four broad categories of service, shown in Figure 1: an ingestion layer that accepts incoming device data (often over MQTT or HTTPS, Module 2), a device-management layer, a storage layer, and an analytics/application layer that surfaces the data to dashboards and other consumers.

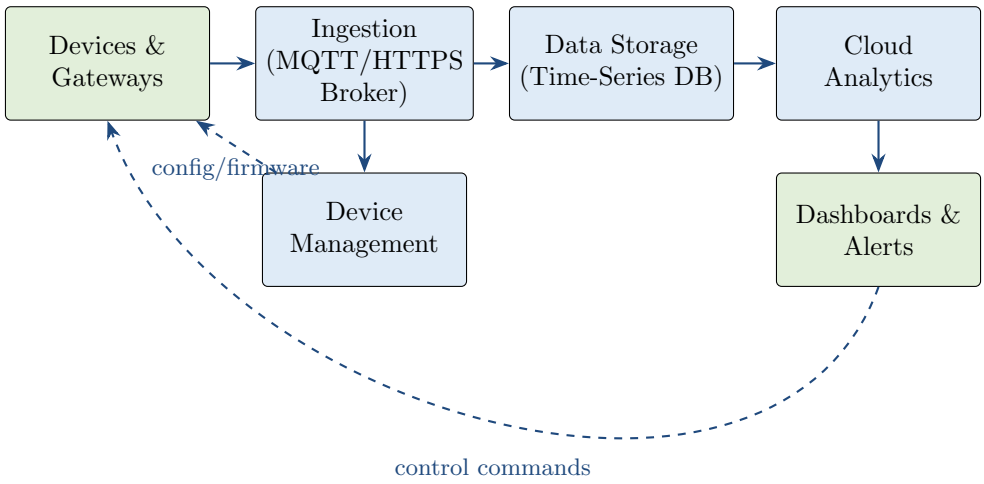


Figure 1: A typical IoT cloud architecture. Device data is ingested, stored in a time-series database, analysed, and surfaced through dashboards; a separate device-management path handles configuration, firmware updates, and control commands sent back to devices.

3.2 Device Management

At cloud scale, a deployment may involve thousands or millions of individual devices, each needing to be provisioned, authenticated, monitored for health/connectivity, and occasionally updated. **Device management** services handle this life cycle centrally – issuing device identities and credentials, tracking which devices are online, and pushing firmware or configuration updates – rather than requiring this to be done manually, device by device.

3.3 Data Storage

IoT data is overwhelmingly **time-series** in nature: a continuous stream of timestamped readings from each device. Cloud platforms typically use storage systems optimised specifically for this pattern – efficient at high-volume sequential writes and at range queries over time ("give me every reading from sensor 42 in the last 24 hours") – rather than a general-purpose relational database, which would perform poorly at this scale and access pattern.

3.4 Cloud Analytics and Dashboards

Cloud analytics services run aggregation, statistical analysis, and machine-learning models (Section 5) over the accumulated data, drawing on far more historical data and computational power than any single edge device could hold. The results are typically surfaced through **IoT dashboards** – web or mobile interfaces showing live and historical data, alerts, and controls – which is how most end users, from a homeowner to a factory operations manager, actually interact with an IoT deployment.

4 Edge Computing

4.1 The Need for Edge Computing

Sending every raw reading to the cloud for processing, as Section 3 implicitly assumes, becomes impractical as a deployment grows: network bandwidth is consumed proportionally to device count and sampling rate, round-trip latency to a distant data centre can be far too slow for time-critical control, and continuous cloud connectivity cannot always be guaranteed. **Edge computing** addresses this by performing computation *directly on, or very near, the device that generated the data*, rather than transmitting every reading onward for central processing.

4.2 Edge versus Cloud

Aspect	Edge Computing	Cloud Computing
Location	At or very near the device	Centralised data centre
Latency	Very low (local)	Higher (network round-trip)
Processing power	Limited by device hardware	Effectively elastic
Bandwidth use	Low (only summaries sent onward)	High (raw data transmitted)
Historical context	Limited to local/recent data	Full deployment history available
Connectivity dependence	Can operate offline	Requires network connectivity

Table 2: Edge computing versus cloud computing.

4.3 Real-Time Processing and Fog Computing

Applications with hard timing requirements – an industrial safety interlock, a collision-avoidance system – cannot tolerate the round-trip latency of a cloud request, making local, **real-time processing** at the edge a functional requirement rather than an optimisation. **Fog computing**, introduced briefly in Module 1, provides an intermediate tier between the edge and the cloud: a local gateway or on-premises server aggregates and processes data from many nearby edge devices, absorbing much of the bandwidth and latency cost before a smaller, filtered volume is sent onward to the cloud for longer-term storage and analytics.

4.4 Edge AI

Edge AI refers specifically to running trained machine-learning models directly on edge hardware – for example, a lightweight anomaly-detection model on an ESP32, or a computer-vision model on a Raspberry Pi – rather than sending data to the cloud for inference. This is what allows the

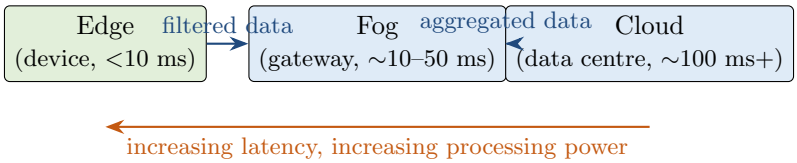


Figure 2: The edge–fog–cloud processing spectrum. Moving from edge to cloud trades lower latency and bandwidth use for greater processing power and historical context.

"Edge AI" stage of the example pipeline in Section 6 to flag an unusual reading within milliseconds, without waiting on a network round trip.

Worked Example

Edge versus cloud decision. A conveyor-belt system must stop within 20 ms of detecting an obstruction to avoid damaging goods, but the network link to the nearest cloud region has a typical round-trip latency of 120 ms. Should obstruction detection be implemented at the edge or in the cloud, and why?
Solution. At the edge. The safety requirement (react within 20 ms) is far tighter than the network round-trip latency to the cloud (120 ms), so any cloud-dependent design would already have missed the deadline before a response could even arrive; only local, edge-based processing can meet a 20 ms reaction requirement.

5 IoT + AI

5.1 Machine Learning with Sensor Data

IoT deployments generate exactly the kind of large, continuous, labelled-or-labellable data stream that machine-learning models are trained on: temperature curves, vibration signatures, image frames, and so on. Applying machine learning to this data – whether trained in the cloud on historical data and then deployed to the edge (Section 4.4), or run entirely in the cloud – is what turns a raw sensor stream into actionable insight rather than just a logged number.

5.2 Anomaly Detection

Anomaly detection models learn the normal pattern of a sensor's readings and flag deviations from that pattern – an unusual spike in vibration, a sudden temperature drop outside the expected daily cycle – often before a simple fixed threshold would trigger, and without needing to be told in advance exactly what a fault looks like.

5.3 Predictive Maintenance

Predictive maintenance extends anomaly detection one step further: rather than simply flagging that a machine's current readings are unusual, a model trained on historical sensor data leading up to past failures can estimate the *probability* or *expected time* of a future failure, allowing maintenance to be scheduled proactively before a breakdown occurs, rather than reactively after one.

5.4 Computer Vision

Where a deployment includes a camera module, **computer vision** models extract structured information directly from image or video frames – detecting whether a parking space is occupied, whether a worker is wearing required safety equipment, or counting vehicles at an intersection – turning visual data into the same kind of structured, actionable signal that a numeric sensor reading provides.

5.5 Towards Intelligent IoT

Taken together, these techniques move a deployment from simple threshold-based monitoring toward what is often called **Intelligent IoT**: a system that not only reports what is happening, but anticipates what is likely to happen next and can, in many cases, act on that prediction automatically. This is the capability Unit V's applications rely on most heavily – disease detection in smart agriculture, ECG-based emergency alerts in healthcare, and predictive maintenance in industrial IoT are all direct applications of the techniques introduced in this section.

6 The Complete Edge-to-Cloud Pipeline

Sections 2 through 5 can now be assembled into a single, representative pipeline that recurs, in specialised form, throughout Unit V:

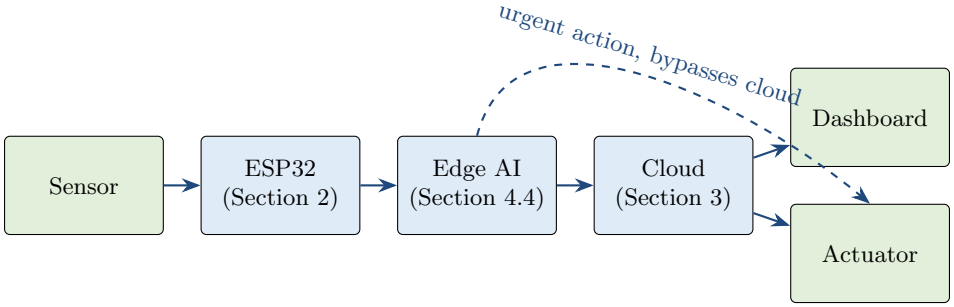


Figure 3: The example edge-to-cloud IoT pipeline: Sensor → ESP32 → Edge AI → Cloud → Dashboard/Actuator. A time-critical response can act directly from the Edge AI stage (dashed path), bypassing the cloud round trip entirely, while routine data continues on to the cloud for storage, analytics, and dashboard display.

Worked Example

Tracing the pipeline. A vibration sensor on an industrial motor feeds an ESP32, which runs a small anomaly-detection model locally. Under normal operation, readings are forwarded to the cloud every minute for long-term trend analysis and dashboard display. If the local model detects a sudden extreme vibration spike consistent with imminent bearing failure, the motor's power relay is cut immediately. Identify which stage of Figure 3 handles the emergency shutdown, and explain why this stage – rather than the cloud – handles it.

Solution. The Edge AI stage handles the emergency shutdown, acting directly on the actuator (the dashed path in Figure 3) rather than routing through the cloud. This mirrors the edge-versus-cloud latency argument of Section 4.5: an imminent bearing failure requires an immediate response, and waiting for a round trip to the cloud would risk the failure occurring

before a shutdown command could return.

IoT in Practice: Connecting Back to the Course

This module completes the "how is it built" half of the course. Modules 1 and 2 supplied the devices and communication links; this module supplies the platforms they run on and the edge-to-cloud spectrum their data flows through. Unit IV builds directly on Section 3's cloud architecture, adding the security controls needed to protect device identities, data in transit, and cloud storage, and extends Section 5's machine-learning techniques into a fuller treatment of IoT data analytics. Unit V then shows this exact Sensor → Platform → Edge → Cloud → Dashboard/Actuator pipeline instantiated across seven application domains, with the balance of edge versus cloud processing, and the specific AI technique applied, tailored to each domain's latency and data characteristics.

7 Solved Problems

Worked Example

Problem 1. A student wants to build a battery-powered outdoor temperature logger that only needs to store readings locally on an SD card for later retrieval, with no network connectivity required at all. Which hardware platform from Table 1 is the most power-efficient appropriate choice, and why is a Raspberry Pi unnecessarily heavy for this task?

Solution. A basic Arduino board is the most power-efficient appropriate choice: the task requires no network connectivity and only simple control logic (reading a sensor, writing to an SD card), which Arduino's low-power, single-program execution model handles well. A Raspberry Pi's full operating system and higher power draw would shorten battery life substantially for no functional benefit in this offline, low-complexity task.

Worked Example

Problem 2. Explain, using Section 3.3, why an IoT cloud platform typically uses a time-series database rather than a general-purpose relational database for storing sensor readings.

Solution. IoT sensor data consists of very high-volume, sequential, timestamped writes, and is predominantly queried by time range for a given device; time-series databases are purpose-built to handle exactly this write and query pattern efficiently at scale, whereas a general-purpose relational database, optimised for more varied transactional workloads, would perform comparatively poorly under this specific pattern.

Worked Example

Problem 3. A smart building has 2,000 environmental sensors, each producing a reading every 10 seconds. Explain, referencing Section 4.1, why routing every single reading to the cloud for processing becomes increasingly impractical as sensor count grows, and name the architectural tier that mitigates this.

Solution. At 2,000 sensors reporting every 10 seconds, the aggregate data rate and required cloud bandwidth scale directly with device count, and this cost only grows as more sensors are added; fog computing mitigates this by aggregating and pre-processing data from many local sensors at a nearby gateway, sending only a smaller, filtered volume onward to the cloud rather

than every raw reading.

Worked Example

Problem 4. Distinguish, in one or two sentences each, anomaly detection from predictive maintenance as applied to a factory machine's vibration data.
Solution. Anomaly detection identifies that the *current* vibration reading deviates from the machine's normal pattern, without necessarily predicting what will happen next. Predictive maintenance goes further, using historical data leading up to past failures to estimate the likelihood or expected timing of a *future* failure, enabling maintenance to be scheduled proactively rather than triggered only after an anomaly is already occurring.

8 Summary

IoT computation is distributed across a spectrum of hardware platforms and processing tiers. Arduino, ESP32, NodeMCU, and Raspberry Pi trade off connectivity, processing power, and cost, from simple fixed-program microcontrollers to full Linux-based single-board computers. Cloud platforms provide device management, time-series storage, and analytics services at a scale no single device could match, surfaced to end users through dashboards. Edge computing pushes processing back toward the device to reduce latency and bandwidth use, with fog computing serving as an intermediate aggregation tier, and Edge AI extending this to running trained machine-learning models locally for near-instant response. Machine learning applied to sensor data – anomaly detection, predictive maintenance, and computer vision – is what ultimately turns this edge-to-cloud pipeline into the anticipatory, semi-autonomous behaviour associated with Intelligent IoT, and recurs, tailored to each domain, throughout Unit V's applications.

References

1. Course syllabus, *Internet of Things (IoT) with Applications* — Unit III: IoT Platforms, Edge and Cloud.
2. W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, 2016.
3. A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, 2015.
4. Course notes, *Internet of Things (IoT) with Applications* – Module 3, IoT Platforms, Edge and Cloud.